

Memoria Práctica Final

Proyecto Data Warehouse en Snowflake

MÓDULO

Entornos Cloud

HERRAMIENTA

Snowflake

PARTES COMPLETADAS

A (Dataset) hasta G (Reprocesado)

ESTADO ACTUAL

Base de datos terminada. Preparado para cargar en Power BI.

PARTES A Y B: DATASET Y CAPA RAW

Para empezar el proyecto, he usado el dataset de prueba **TPCH_SF1** que viene en Snowflake. En esta primera capa (RAW o zona de aterrizaje) simplemente he copiado las tablas tal cual vienen para tener una copia exacta de los datos originales.

```
-- Creación de Base de Datos y Esquema
CREATE DATABASE IF NOT EXISTS PRACTICA_FINAL_DAW;
USE DATABASE PRACTICA_FINAL_DAW;

CREATE SCHEMA IF NOT EXISTS RAW;
USE SCHEMA RAW;

-- Clonación de tablas desde el Dataset de origen
CREATE OR REPLACE TABLE RAW_ORDERS AS SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
CREATE OR REPLACE TABLE RAW_LINEITEM AS SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM;
CREATE OR REPLACE TABLE RAW_PART AS SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.PART;
CREATE OR REPLACE TABLE RAW_CUSTOMER AS SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER;
CREATE OR REPLACE TABLE RAW_NATION AS SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.NATION;
CREATE OR REPLACE TABLE RAW_REGION AS SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.REGION;
```

Generación de Datos Nuevos y Corrección de Errores

Como pedía el enunciado, tuve que crear archivos CSV a mano para las tablas nuevas: Tienda, Promoción y Cambio. Al intentar subir las promociones que generé con la web Mockaroo, me dio un error de lectura. En lugar de borrar y volver a subir todo, lo arreglé directamente desde Snowflake borrando la columna que estaba mal y recalculando la fecha de fin de la promoción sumando días al azar.

```
-- Arreglo del error de fechas en la tabla de promociones
ALTER TABLE RAW_PROMOCION DROP COLUMN FECHA_FIN;
ALTER TABLE RAW_PROMOCION ADD COLUMN FECHA_FIN DATE;
UPDATE RAW_PROMOCION
SET FECHA_FIN = DATEADD(day, UNIFORM(3, 15, RANDOM()), TRY_CAST(FECHA_INICIO AS DATE));
```

Añadiendo horas aleatorias y zonas horarias

Otra cosa que hice en esta capa fue prepararlo todo para el cruce de horas. Le añadí a cada país su zona horaria correcta, y a los pedidos (que solo traían la fecha) les añadí una hora y un ID de tienda de forma aleatoria para simular datos de ventas reales.

```
-- Añadir columnas nuevas a los pedidos
ALTER TABLE RAW_ORDERS ADD COLUMN O_ORDERDATETIME TIMESTAMP;
ALTER TABLE RAW_ORDERS ADD COLUMN O_STOREKEY NUMBER;

-- Poner horas aleatorias y una tienda del 1 al 10 al azar
UPDATE RAW_ORDERS
SET
    O_ORDERDATETIME = DATEADD(SECOND, UNIFORM(0, 24*60*60-1, RANDOM()), TO_TIMESTAMP(O_ORDERDATE)),
    O_STOREKEY = UNIFORM(1, 10, RANDOM());
```

PARTE C: CAPA STAGING (LIMPIEZA)

Para la capa de Staging he decidido usar **VISTAS (VIEWS)**. He elegido esto porque así no tenemos que guardar los mismos datos repetidos en el disco y gastar espacio en Snowflake. Las vistas me sirven como un "filtro" para limpiar cosas, por ejemplo, quitar la palabra "Clerk#" de los empleados para quedarnos solo con su número.

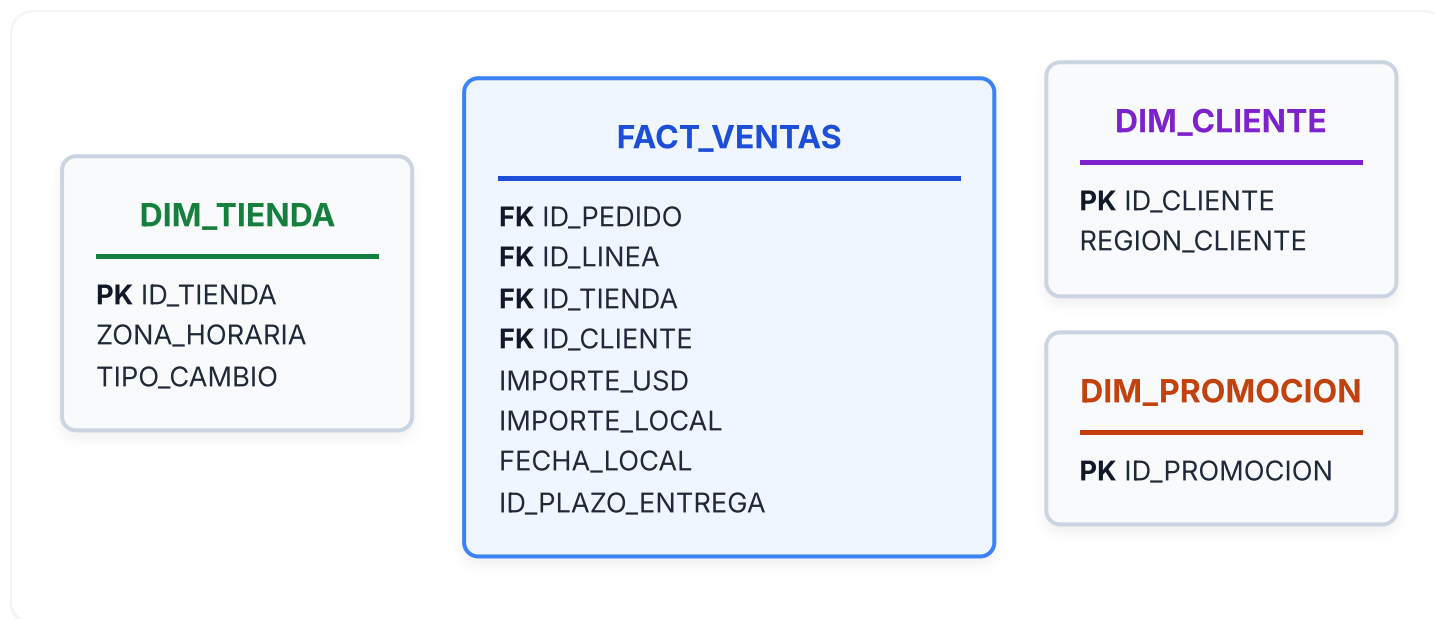
```
CREATE SCHEMA IF NOT EXISTS STAGING;
USE SCHEMA STAGING;

-- Limpieza del campo O_CLERK para quedarnos solo con el número
CREATE OR REPLACE VIEW STG_ORDERS AS
SELECT
    O_ORDERKEY, O_CUSTKEY, O_ORDERSTATUS, O_TOTALPRICE, O_ORDERDATE,
    CAST(REPLACE(O_CLERK, 'Clerk#', '')) AS NUMBER) AS O_CLERK_ID,
    O_ORDERDATETIME, O_STOREKEY
FROM PRACTICA_FINAL_DAW.RAW.RAW_ORDERS;
```

PARTE D: CAPA WAREHOUSE (EL MODELO)

He montado un **Modelo en Estrella (Star Schema)**, ya que es el formato más cómodo y rápido para enchufarlo después a Power BI. Como pedía el enunciado, he usado un SCD Tipo 1, es decir, no guardamos el historial de cambios y usamos las claves originales de los clientes y tiendas. La tabla central (de hechos) hace todo el trabajo duro de convertir monedas y ajustar las horas según el país usando subconsultas en el FROM.

MI MODELO EN ESTRELLA



```
CREATE OR REPLACE TABLE FACT_VENTAS AS
SELECT
    base.ID_PEDIDO, base.ID_LINEA, base.ID_ARTICULO, base.ID_TIENDA, base.ID_CLIENTE,
    base.ID_PROMOCION, base.TIPO_OPERACION, base.UNIDADES, base.IMPORTE_USD,
    -- Convierto a la moneda local y aplico la zona horaria
    ROUND(base.IMPORTE_USD * dt.TIPO_CAMBIO_TIENDA, 2) AS IMPORTE_MONEDA_TIENDA,
    CONVERT_TIMEZONE('UTC', dc.ZONA_HORARIA_CLIENTE, base.FECHA_HORA_PEDIDO.UTC) AS FECHA_HORA_LOCAL,
    -- Saco el plazo de entrega calculando días de diferencia
    CASE
        WHEN DATEDIFF(day, base.FECHA_ESTIMADA, base.FECHA_ENTREGA) <= 0 THEN 1 -- En plazo
        WHEN DATEDIFF(day, base.FECHA_ESTIMADA, base.FECHA_ENTREGA) <= 10 THEN 2 -- Entrega tardía
        ELSE 0 -- Fuera de plazo
    END AS ID_PLAZO_ENTREGA
FROM (
    -- Subconsulta de la que tiro para cruzar luego todo
    SELECT l.L_ORDERKEY AS ID_PEDIDO, l.L_LINENUMBER AS ID_LINEA...
) AS base
LEFT JOIN DIM_TIENDA dt ON base.ID_TIENDA = dt.ID_TIENDA
LEFT JOIN DIM_CLIENTE dc ON base.ID_CLIENTE = dc.ID_CLIENTE;
```

PARTE E: CAPA ANALYTICS Y SEGURIDAD

La capa de Analytics es la que le voy a enseñar al cliente final. Como el profesor pedía que el cliente no pudiese ver todo el código ni meterse en las capas anteriores (RAW, Staging, etc.), le he creado un Rol específico y he usado **Vistas Seguras (Secure Views)**. Así, el cliente ve los datos pero el código SQL está oculto.

```
-- Crear un rol para el usuario de negocio y darle permisos solo aquí
CREATE ROLE IF NOT EXISTS ROL_ANALISTA;
GRANT USAGE ON DATABASE PRACTICA_FINAL_DAW TO ROLE ROL_ANALISTA;
GRANT USAGE ON SCHEMA PRACTICA_FINAL_DAW.ANALYTICS TO ROLE ROL_ANALISTA;
GRANT SELECT ON ALL VIEWS IN SCHEMA PRACTICA_FINAL_DAW.ANALYTICS TO ROLE ROL_ANALISTA;
```

Aquí dejo un ejemplo de un par de consultas que he dejado preparadas en esta capa, usando las monedas y horas que calculé en Warehouse:

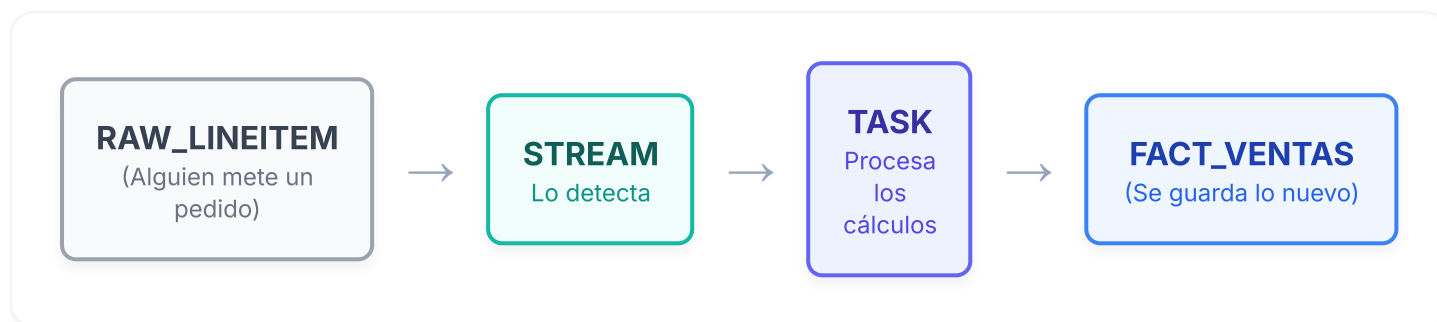
```
-- Consulta 1: Rendimiento por Tienda (con moneda de la tienda)
CREATE OR REPLACE SECURE VIEW V_ANALISIS_TIENDA AS
SELECT
    dt.PAIS_TIENDA,
    dt.MONEDA_TIENDA,
    fv.TIPO_OPERACION,
    COUNT(fv.ID_PEDIDO) AS TOTAL_OPERACIONES,
    SUM(fv.IMPORTE_MONEDA_TIENDA) AS TOTAL_IMPORTE_LOCAL
FROM PRACTICA_FINAL_DAW.WAREHOUSE.FACT_VENTAS fv
JOIN PRACTICA_FINAL_DAW.WAREHOUSE.DIM_TIENDA dt ON fv.ID_TIENDA = dt.ID_TIENDA
GROUP BY dt.PAIS_TIENDA, dt.MONEDA_TIENDA, fv.TIPO_OPERACION;

-- Consulta 5: Tendencia de ingresos al mes (con hora del cliente)
CREATE OR REPLACE SECURE VIEW V_TENDENCIA_TEMPORAL AS
SELECT
    DATE_TRUNC('MONTH', fv.FECHA_HORA_LOCAL_CLIENTE) AS MES_VENTA,
    dc.REGION_CLIENTE,
    SUM(fv.IMPORTE_USD) AS INGRESOS_USD
FROM PRACTICA_FINAL_DAW.WAREHOUSE.FACT_VENTAS fv
JOIN PRACTICA_FINAL_DAW.WAREHOUSE.DIM_CLIENTE dc ON fv.ID_CLIENTE = dc.ID_CLIENTE
WHERE fv.TIPO_OPERACION = 'Venta'
GROUP BY DATE_TRUNC('MONTH', fv.FECHA_HORA_LOCAL_CLIENTE), dc.REGION_CLIENTE
ORDER BY MES_VENTA DESC;
```

PARTE F: SINCRONIZACIÓN (NUEVOS REGISTROS)

El enunciado pide que si metemos un registro nuevo en la tabla inicial (RAW), tiene que llegar él solo a la capa Analytics. Para automatizar esto en Snowflake sin gastar mucho dinero en ejecutar códigos a todas horas, he creado un **STREAM** que vigila la tabla de líneas de pedido, y una **TASK** que solo se enciende cuando el Stream le avisa de que hay algo nuevo.

CÓMO FLUYEN LOS DATOS SOLOS



```
-- 1. Stream (Se queda mirando la tabla RAW)
CREATE OR REPLACE STREAM PRACTICA_FINAL_DAW.RAW.STREAM_NUEVAS_VENTAS
ON TABLE PRACTICA_FINAL_DAW.RAW.RAW_LINEITEM;

-- 2. La Task (Solo arranca si el Stream le dice que hay datos)
CREATE OR REPLACE TASK PRACTICA_FINAL_DAW.WAREHOUSE.TASK_ACTUALIZAR_FACT_VENTAS
WAREHOUSE = COMPUTE_WH
WHEN SYSTEM$STREAM_HAS_DATA('PRACTICA_FINAL_DAW.RAW.STREAM_NUEVAS_VENTAS')
AS
INSERT INTO PRACTICA_FINAL_DAW.WAREHOUSE.FACT_VENTAS
SELECT /* Cálculos y conversiones */ ...
FROM (
  -- Aquí cogemos solo las filas NUEVAS que han entrado al stream
  SELECT * FROM PRACTICA_FINAL_DAW.RAW.STREAM_NUEVAS_VENTAS s
  WHERE s.METADATA$ACTION = 'INSERT' AND s.METADATA$ISUPDATE = FALSE
) AS base;

-- Importante: Encender la task porque en Snowflake se crean apagadas
ALTER TASK PRACTICA_FINAL_DAW.WAREHOUSE.TASK_ACTUALIZAR_FACT_VENTAS RESUME;
```

PARTE G: REPROCESADO (CAMBIO DE NORMAS)

En esta parte de la práctica me pedían simular que la empresa ha cambiado las reglas de negocio, metiendo el estado "Entrega Crítica" (de 10 a 30 días). Este cambio me ha obligado a modificar el código en tres sitios distintos para que no se rompa nada:

- **Tabla de Hechos (Pasado):** Tuve que hacer un CREATE OR REPLACE para borrar y volver a cargar todo el histórico con la nueva norma.
- **Analytics (Presente):** Tuve que modificar la vista que lee esto para añadirle el texto "Entrega Crítica" al código 3.
- **Task (Futuro):** Tuve que pausar la tarea, cambiarle la condición dentro del código, y volverla a encender para que los pedidos nuevos salgan bien calculados.

Así ha quedado la nueva condición de envíos:

```
CASE
  WHEN DATEDIFF(day, base.FECHA_ESTIMADA, base.FECHA_ENTREGA) <= 0 THEN 1 -- En plazo
  WHEN DATEDIFF(day, base.FECHA_ESTIMADA, base.FECHA_ENTREGA) <= 10 THEN 2 -- Entrega ta
  WHEN DATEDIFF(day, base.FECHA_ESTIMADA, base.FECHA_ENTREGA) <= 30 THEN 3 -- AÑADIDO: E
  ELSE 0 -- Modificado: Fuera de plazo (ahora es más de 30)
END AS ID_PLAZO_ENTREGA
```

Estado Final del Proyecto

Con esto he dado por terminada toda la parte de bases de datos y SQL en **Snowflake**. Todo el proceso que va desde que llegan los datos brutos, se limpian, se cruzan y se preparan las consultas finales, funciona de manera automática y segura.

Lo que me falta para terminar la práctica (Partes H e I):



Conectar esto a Power BI y crear un panel con 6 gráficos.



Mirar la herramienta Query Profile de Snowflake para hacer un reporte de cuánto tardan en cargar las consultas.